

Estructuras de Datos II

Trabajo Práctico 2

Jerónimo Delorenzi, Valentino Dominguez, Bruno Minighin

2026

Item b

Para la implementación dada en **a**, dar la especificación de costos (trabajo y profundidad) de las funciones `filterS`, `reduceS` y `scanS`.

1. `filterS`

Implementación.

```
filterS _ [] = []
filterS p (x:xs) = let
    (px, pxs) = p x || filterS p xs
  in
    if px then x:pxs else pxs
```

Trabajo.

Para analizar el trabajo de `filterS` consideremos los distintos casos de la implementación.

- Como en el caso base solo consiste en devolver la lista vacía, el trabajo es

$$W(\text{filterS } p []) = c_1$$

que donde $c_1 = O(1)$.

- En cada caso recursivo, se ejecutan p evaluando el primer elemento de la lista, y luego el llamado recursivo con el resto de esta última. Por último, se forma la nueva lista si se verifica el predicado para los resultados obtenidos.

Entonces, para obtener el costo del llamado recursivo necesitamos notar el costo constante del `let/in` y del `if`, la evaluación del predicado

y de la llamada recursiva. Entonces

$$W(\text{filterS } p \ (x : xs)) = c_2 + W(p \ x) + W(\text{filterS } p \ xs)$$

Si desarrollamos recursivamente la recurrencia y suponiendo que $|xs|$ es el tamaño de la secuencia obtenemos

$$\begin{aligned} W(\text{filterS } p \ (x : xs)) &= W(p \ x) + W(p \ x_1) + \dots + W(p \ x_{n-1}) + n \cdot c_2 + c_1 \\ &= n \cdot c_2 + c_1 + \sum_{i=0}^{|xs|-1} W(p \ x_i) \end{aligned}$$

Por lo tanto podemos concluir que

$$W(\text{filterS } p \ (x : xs)) \in O \left(|xs| + \sum_{i=0}^{|xs|-1} W(p \ x_i) \right)$$

Profundidad.

De una forma similar en el desarrollo, pero esta vez teniendo en cuenta que en el caso recursivo $p \ x$ y $\text{filterS } p \ xs$ se paraleliza esto último, tenemos

- $S(\text{filterS } p \ []) = c_1$
- $S(\text{filterS } p \ (x : xs)) = c_2 + \max(S(p \ x), S(\text{filter } p \ xs))$ donde si desarrollamos la recurrencia obtenemos que

$$S(\text{filterS } p \ (x : xs)) \in O \left(|xs| + \max_{i=0}^{|xs|-1} S(p \ x_i) \right)$$

2. reduceS

Implementación.

```
reduceS _ e [] = e
reduceS op e [x] = op e x
reduceS op e xs = reduceS op e (contraer op xs)
  where
    contraer :: (a → a → a) → [a] → [a]
    contraer op [] = []
    contraer op [x] = [x]
    contraer op (x:y:xs) = let (res, rest) = op x y ||| contraer
                           op xs in res : rest
```

Trabajo.

Primero, calculemos el trabajo de contraer.

- El costo de devolver la lista vacía claramente es $c_1 = O(1)$.
- El costo de devolver la lista con un elemento es $c_2 = O(1)$.
- En cada caso recursivo, se ejecuta $\oplus$ evaluando el primer elemento de la lista, y luego el llamado recursivo con el resto de esta última. Al final, juntamos los resultados obtenidos.

Entonces, para obtener el costo del llamado recursivo necesitamos notar el costo constante del **let/in** y del operador $(:)$. Luego,

$$W(\text{contraer} \oplus (x_1 : x_2 : xs)) = W(x \oplus y) + W(\text{contraer} \oplus xs)$$

$$W(x_1 \oplus x_2) + W(x_3 \oplus x_4) + \dots + W(x_{n-1} \oplus y_{n-1}) + \frac{|xs|}{2} \cdot c_3 + c_1$$

Con lo cual,

$$W(\text{contraer} \oplus (x_1 : x_2 : xs)) \in O \left(|xs| + \sum_{i=0}^{|xs|-1} W(x_{2i} \oplus x_{2i+1}) \right)$$

Sabiendo esto, podemos obtener el costo del trabajo de **reduceS**.

- $W(\text{reduceS} \oplus e []) = c_1$.
- $W(\text{reduceS} \oplus e [x]) = c_2$
- $W(\text{reduceS} \oplus e s) =$
 $W(\text{contraer} \oplus s) + W(\text{reduceS} \oplus e (\text{contraer} \oplus xs)) + c_3$
 En cada contracción vamos operando de dos en dos elementos, si lo pensamos como un árbol de contracción tendremos $\lg(n)$ niveles de recursión, esto debido a que luego de realizar la operación, **contraer** devuelve una secuencia de tamaño aproximado $\frac{|s|}{2}$. Por lo tanto,

$$W(\text{reduceS} \oplus e s) \in O \left(|s| + \sum_{(x \oplus y) \in O_r(\oplus, b, s)} W(x \oplus y) \right)$$

Profundidad.

De igual forma, con un razonamiento similar podemos llegar a los siguientes resultados. Primero veamos la profundidad de **contraer**.

- $S(\text{contraer} \oplus []) = c_1$
- $S(\text{contraer} \oplus [x]) = c_2$
- $S(\text{contraer} \oplus (x_1 : x_2 : xs)) = \max(S(x_1 \oplus x_2), S(\text{contraer} \oplus xs)) + c_3$.
 Mencionamos que cuando calculamos el trabajo que el árbol resulta

de $\lg(n)$ niveles. Como la operación sigue siendo recursiva por el resto de la lista, el costo de **contraer** sigue siendo lineal en $|xs|$. Entonces

$$S(\text{contraer} \oplus (x_1 : x_2 : xs)) \in O \left(|xs| + \max_{i=0}^{|xs|-1} S(x_{2i} \oplus x_{2i+1}) \right)$$

Para la profundidad

- $S(\text{reduceS} \oplus e []) = c_1$
- $S(\text{reduceS} \oplus e [x]) = c_2 + S(e \oplus x)$
- $S(\text{reduceS} \oplus e s) =$
 $S(\text{contraer} \oplus s) + S(\text{reduceS} \oplus e (\text{contraer} \oplus xs)) + c_3$

Al igual que en el Trabajo, el llamado a **contraer** reduce el tamaño de la lista a la mitad en cada paso, generando un árbol de llamadas de altura $\lg(|s|)$. Como en cada nivel tenemos un costo lineal $|xs|$ para recorrer la lista, la profundidad total resulta en:

$$S(\text{reduceS} \oplus e s) \in O \left(|s| + \lg |s| \max_{(x \oplus y) \in O_r(\oplus, e, s)} S(x \oplus y) \right)$$

3. scanS

Implementación

```
scanS op e [] = ([], e)
scanS op e [x] = ([e], op e x)
scanS op e xs = let
    (l, r) = scanS op e (contraer op xs)
    in (expandir op xs l True, r)
where
    contraer :: (a → a → a) → [a] → [a]
    contraer op [] = []
    contraer op [x] = [x]
    contraer op (x:y:xs) = let
        (res, rest) = op x y ||| contraer op xs
        in res : rest

    expandir :: (a → a → a) → [a] → [a] → Bool → [a]
    expandir op _ [] _ = []
    expandir op [x] (y:ys) p | p = [y]
    | otherwise = [op y x]
    expandir op (x1:x2:xs) (y:ys) p | p = y : expandir op (x1:x2:xs) (y:ys) (not p)
    | otherwise =
        let
            (res, rest) = op y x1 ||| expandir op xs ys (
                not p)
        in res: rest
```

Trabajo.

Ya sabemos el trabajo y profundidad de `contraer`. Ahora calculamos la de `expandir`

- El costo de devolver la lista vacía es constante, entonces
 $W(\text{expandir} \oplus [] _) = c_1$.
- $W(\text{expandir} \oplus [x] (y : ys) p) = c_2$ ya que puede devolver una lista de un elemento o la operación entre x e y , entonces, también es constante.
- $W(\text{expandir} \oplus (x1 : x2 : xs) (y : ys) \text{True})$, como la condición es verdadera, agregamos el primer elemento del acumulador a la llamada recursiva de `expandir`. Entonces
 $W(\text{expandir} \oplus (x1 : x2 : xs) (y : ys) \text{True})$
 $= W(\text{expandir} \oplus (x1 : x2 : xs) (y : ys) \text{False}) + c_3$
- $W(\text{expandir} \oplus (x1 : x2 : xs) (y : ys) \text{False})$, como la condición es falsa, realizamos la operación con el elemento de la secuencia junto con el elemento del acumulador junto con la llamada recursiva para luego realizar la operación $:$. Tenemos
 $W(\text{expandir} \oplus (x1 : x2 : xs) (y : ys) \text{False})$
 $= W(x \oplus y) = W(\text{expandir} \oplus xs \ y \ \text{True}) + c_4$.

Por lo tanto, juntando los últimos casos para cada par de elemento de la secuencia se realiza $\oplus$

$$W(\text{expandir} \oplus xs \ ys \ p) \in O \left(|xs| + \sum_{i=0}^{|xs|-1} W(y_i \oplus x_{2i}) \right)$$

Para `scanS`

- $W(\text{scanS} \oplus e []) = c_1$
- $W(\text{scanS} \oplus e [x]) = W(e \oplus x) + c_2$
- $W(\text{scanS} \oplus e \ s) =$
 $W(\text{contraer} \oplus s) + W(\text{scanS} \oplus e (\text{contraer} \oplus s)) +$
 $W(\text{expandir} \oplus s \ l \ \text{True}) + c_3$

De igual manera que en `reduceS`, la suma de todos los $W(x \oplus y)$ en todos los niveles de contracción y expansión resulta dependiente de la operación. Por lo que

$$W(\text{scanS} \oplus e \ s) \in O \left(|s| + \sum_{(x \oplus y) \in O_s(\oplus, b, s)} W(x \oplus y) \right)$$

Profundidad.

Rápidamente, resumimos los costos de la profundidad de **expandir** como

- $S(\text{expandir} \oplus - [] -) = c_1$
- $S(\text{expandir} \oplus [x] (y : ys) p) = c_2$
- $S(\text{expandir} \oplus (x1 : x2 : xs) (y : ys) \text{True}) =$
 $S(\text{expandir} \oplus (x1 : x2 : xs) (y : ys) \text{False}) + c_3$
- $S(\text{expandir} \oplus (x1 : x2 : xs) (y : ys) \text{False}) =$
 $\max(S(y \oplus x1), S(\text{expandir} \oplus xs ys \text{True})) + c_4$

Aunque tenemos una paralelización, la recursión sigue siendo secuencial al realizarse sobre listas. Por lo tanto

$$S(\text{expandir} \oplus xs ys p) \in O \left(|xs| + \max_{i=0}^{|xs|-1} S(y_i \oplus x_{2i}) \right)$$

Con respecto a **scanS**,

- $S(\text{scanS} \oplus e []) = c_1$
- $S(\text{scanS} \oplus e [x]) = c_2 + S(e \oplus x)$
- $S(\text{scanS} \oplus e s) =$
 $S(\text{contraer} \oplus s) + S(\text{scanS} \oplus e (\text{contraer} \oplus s)) +$
 $S(\text{expandir} \oplus s l \text{True}) + c_3$

Dado que **scanS** llama recursivamente a **contraer**, para luego llamarse a él y por último a **expandir**, al ser secuenciales aportan un término lineal en cada nivel.

Como explicamos en **reduceS**, se reduce el tamaño de la lista a la mitad en cada paso, generando un árbol de llamadas de altura $\lg(|s|)$. Por lo tanto,

$$S(\text{scanS} \oplus e s) \in O \left(|s| + \lg |s| \max_{(x \oplus y) \in O_r(\oplus, e, s)} S(x \oplus y) \right)$$

Item d

Muestre que la implementación dada en **c** verifica la especificación de costos para la profundidad de la operación **reduceS**.

Implementación.

```
reduceS op e xs | len == 0 = e
               | len == 1 = op e (nthS xs 0)
               | otherwise = reduceS op e (A.tabulate contraer mitad)
               where
                 len = lengthS xs
                 mitad = (len+1) `div` 2
                 contraer i | odd len && i == mitad - 1 = nthS xs (2*i)
                           | otherwise = op (nthS xs (2*i)) (nthS xs (2*i + 1))
```

Profundidad.

Supongamos que la operación $\oplus \in O(1)$. Entonces, queremos probar que $O(\text{reduceS } \oplus e s) \in O(\lg |s|)$.

Veamos el costo por casos:

- Cuando la longitud es 0 (caso base 1), devolvemos el valor e , por lo que tenemos costo constante, $S(\text{reduceS } \oplus e \langle \rangle) = c_1$
- Cuando la longitud es 1 (caso base 2), devolvemos la operación entre el valor e y el primer elemento de la secuencia. Para obtener el primer elemento usamos la operación **nthS**, que sabemos que tiene profundidad constante, es decir, $S(\text{nthS } s \ i) \in O(1)$. Luego, por lo dicho previamente y como $\oplus \in O(1)$ tenemos que $S(\text{reduceS } \oplus e \langle x \rangle) = c_2$.
- En el caso recursivo se llama a **reduceS** con la secuencia contraída. Dado este caso, tenemos
$$S(\text{reduceS } \oplus e s) = S(\text{tabulateS contraer } \frac{|s|}{2}) + S(\text{reduceS } \oplus e (\text{tabulateS contraer } \frac{|s|}{2})) + c_3$$

Veamos la profundidad de $S(\text{tabulateS contraer } \frac{|s|}{2})$

- La profundidad de **tabulateS** es $O\left(\max_{i=0}^{|s|-1} S(f \ i)\right)$. La operación **contraer** realiza como máximo dos operaciones **nthS** (que sabemos que es $O(1)$) y una operación $\oplus$ (también $O(1)$) por lo que el máximo de todas será $O(1)$. Entonces

$$S(\text{tabulateS contraer } \frac{|s|}{2}) \in O(1)$$

Por ahora tenemos que $S(\text{reduceS } \oplus e s) = c_4 + S(\text{reduceS } \oplus e (\text{tabulateS contraer } \frac{|s|}{2})) + c_3$, donde $c_4 = O(1)$

La secuencia restante tiene tamaño $\frac{|s|}{2}$. Por lo que en cada llamada recursiva hasta llegar al caso base 2 ocurrirá

$$S(\text{reduceS} \oplus e s) = S(\text{reduceS} \oplus e s') + c_4 \quad (1)$$

$$S(\text{reduceS} \oplus e s) = S(\text{reduceS} \oplus e s'') + 2 \cdot c_4 \quad (2)$$

$$\vdots \quad (3)$$

$$S(\text{reduceS} \oplus e s) = S(\text{reduceS} \oplus e \langle x \rangle) + \lg |s| \cdot c_4(4) \\ = c_4 + \lg |s| \cdot c_4$$

1. s' tiene longitud $\frac{|s|}{2}$ y c_4 abarca a c_3 y al costo del `tabulateS`.
2. s'' tiene longitud $\frac{|s|}{4}$.
3. Después de $\lg |s|$ pasos llegamos al caso base, esto es porque en cada llamada recursiva, la longitud se va acortando por la mitad.
4. Llegamos al caso base.

Por lo tanto, luego de desarrollar la recurrencia, viendo que la longitud de s se va acortando a la mitad en cada llamada recursiva, tenemos que

$$S(\text{reduceS} \oplus e s) \in O(\lg |s|)$$